LLM Agents!

Jon Trnka

2026-03-09

Table of contents

1 Introduction **1**

2 Objective **2**
2.1 What exactly is an LLM Agent? . . . 2
2.2 Agentic AI vs. Non-Agentic AI comparisons + examples . . . 2
2.3 The present and the future: . . . 3

3 Specific Aims **4**
3.1 Main research question: . . . 4

4 Data **4**
4.1 My data will be from 3 different API sources. . . . 4

5 Methods and Research Design **5**
5.1 Experimental Framework . . . 5
5.2 Task Selection . . . 5
5.3 Process: . . . 5
5.4 'Does the complexity of the task predict total taks latency? . . . 7
5.5 'Does the number of tools used predict total task latency? . . . 9
5.6 Does API difficulty predict execution time? . . . 10
5.7 Issues: . . . 12

6 For those wanting more! **12**

7 Questions, comments, suggestions? **12**

8 Disclaimer: **12**

1 Introduction

Hello! My name is Jonathan Trnka and this is my presentation on LLM Agents!

2 Objective

1. Explain what an LLM-Agent is and why its important/relative
2. Go over the the future of LLM-Agents
3. Research question and data to be used
4. Methods and design
5. Discussion
6. Questions?

2.1 What exactly is an LLM Agent?

- To put it simply, an LLM Agent is the next step in evolution of LLMs.
- What is an LLM?
  - As Zhu et al. (2025) states, ". LLM chatbots are designed as reactive conversational systems that receive textual prompts from users, process these inputs using pretrained knowledge, and generate coherent textual responses. Typical use cases include open domain dialogue, customer support, question answering (Zhu et al.,). Despite LLMs being very powerful in their own way, they are very limited by what they can do. Some common issues is outdated information and hallucinated responses.
- What's a hallucinated response you say?
  - A hallucinated response in regards to LLMs is when it creates incorrect, or fabricates content that believes is correct when it is not. For example, you could ask for a 5 step plan and it gives you 4 believe it to be correct. It's all based on the prompt given to it by the user.
- What does the 'agent' part mean?
  - The 'agent' mean corresponds with the autonomous part of an Agnetic LLM. This allows the LLM to resolve tasks on its own, without further input from a user. It can access APIs to access certain cotent, anything on the web, and solve errors it runs into.
- What is RAG?
  - RAG which stands for Retrieval Augmented Generation, is the solution to the limitations of standard LLMs. Ferrag et al. (2025)

2.2 Agentic AI vs. Non-Agentic AI comparisons + examples

2.2.1 Table side-by-side Comparison

Aspect	Agentic AI	Non-Agentic AI
Task Handling	Multi-step planning	Single-shot answer
Tool Use	Dynamic, context-driven	None
Accuracy	High on factual & numeric tasks	Variable; prone to errors
Adaptability	Adjusts plan mid-run	Fixed output

Aspect	Agentic AI	Non-Agentic AI
Error Recovery	Retries or switches tools	No recovery
Transparency	Shows steps & traces	Opaque
Latency	Slower	Fast
Cost	Higher	Low
Best For	Complex tasks	Simple tasks

2.2.2 Types of LLMs and Agentic AI.

Category	Model / System	Description	Agentic?
Standard LLM	GPT-4o / GPT-4 Turbo	General-purpose chat models; strong reasoning but no built-in planning.	No
Standard LLM	Claude 3 Opus / Sonnet	High-quality reasoning and writing; non-agentic unless wrapped in a system.	No
Standard LLM	LLaMA-3.1 / LLaMA-3.2 / LLaMA-3.3	Open models used in research; non-agentic by default.	No
Standard LLM	DeepSeek-V3	Efficient, high-performance model; not agentic without a tool layer.	No
Standard LLM	Mixtral 8x7B / 8x22B	Sparse MoE models; strong performance but no autonomy.	No
Agentic LLM System	OpenAI o-series with function calling	Uses planning + tool calls + multi-step execution when orchestrated.	Yes
Agentic LLM System	Google Gemini 2.0 + Tools	Can plan, call APIs, and execute multi-step tasks when tools are enabled.	Yes
Agentic LLM System	Groq LLaMA-3.3 + your agent pipeline	Becomes agentic through your planning loop, tool calls, and error recovery.	Yes
Agentic LLM System	LangChain Agents	Framework that wraps any LLM with planning, tools, and autonomy.	Yes
Agentic LLM System	Microsoft Autogen Agents	Multi-agent orchestration layer enabling planning and tool use.	Yes
Agentic LLM System	ReAct-style LLM Agents	LLMs using reasoning + action loops to solve tasks step-by-step.	Yes

2.3 The present and the future:

- LLM-Agents are becoming more and more present due to their capabilities, including but not limited to: API integration, self-checking, error correcting, working with other LLM-Agents, saving time, and more!
  - Examples: They can automate tasks like booking appointments, ogranizing files in a certain way, emailing people and so on.
  - In the bigger picture, they are being used for autonomous vehicles, supply chains, logistic agents, healthcare documentation agents.

- As for the future, they will only grow in what they do/can do. They will be able to take on larger tasks, store more memory, work faster, work smarter, better reasoning, better research.

3 Specific Aims

My goal is to evaluate how effective an agentic AI system is compared to a non-agentic baseline. To do this, I will construct two models: an agentic AI capable of multi-step planning, tool use, API integration, and limited error recovery, and a baseline AI that processes only one prompt at a time without tool use or autonomous decision-making. I will benchmark both systems on the same set of tasks and use linear regression techniques to analyze differences in performance, error rates, and execution time.

3.1 Main research question:

"How well can an LLM agent perform multi-step tasks using a real API?"

3.1.1 Sub-research questions:

- Does the complexity of the task predict total task latency?
  - Y = total latency
  - X = task complexity
- Does the type of multi-step task affect planning accuracy?
  - Y = total_latency
  - X = task type
- Does API difficulty predict execution time?
  - Y = total_latency
  - X = API difficulty

4 Data

4.1 My data will be from 3 different API sources.

- Tavily (web search base)
- OpenWeather
- Calculator

5 Methods and Research Design

5.1 Experimental Framework

5.1.1 The study uses a between-systems comparative design. Two AI systems are constructed:

- Agentic AI — a system capable of multi-step planning, tool use, API calls, and limited error recovery. It can autonomously break down tasks, select tools, and retry operations when failures occur.
- Baseline AI — a non-agentic model restricted to single-prompt, single-response interactions. It cannot use tools, plan, or recover from errors. It behaves like a standard chat assistant.
- Both systems are evaluated on identical tasks to isolate the effect of agentic capabilities.

5.2 Task Selection

5.2.1 Tasks are chosen to require reasoning, multi-step execution, or external information retrieval. Each task is designed so that:

- The agentic system can leverage planning and tools.
- The baseline system must attempt the same task without those capabilities.
- Success, failure, and time-to-completion can be measured consistently.

5.2.2 Examples include:

- Multi-step problem solving (e.g., weather + calculation + summarization).
- API-dependent tasks (e.g., retrieving structured data).
- Error-prone tasks where retries matter (e.g., handling missing parameters).

5.3 Process:

5.3.1 Research Planning:

- Identify what my main research question would be and create my sub-research questions. All of which evaluate the performance of an agentic AI vs non-agentic AI (baseline)
- Select and test my choosen variables. task_complexity, api_difficulty, total_latency, tool_count, execution time, number of steps, and error count.

5.3.2 Tool design:

- Chose tools that would be show multi-step agentic behavior. This would be weather retrieval, calculator functions, and web search.
- Gather the required API keys and create a .ENV file. This would allow seamless access when executing code without hard-coding the credentials.

5.3.3 Creation of the Agentic and Baseline 'agents'.

- Create my 'Agents_Multi_step python file to hold all my core functionality. For the agentic AI, this would be: tool definitions, planning logic, benchmarking routines, logging functions, self-check functions, and error-correction strategies.
- I used the same code for my baseline system. The main difference being no planning, no tool use, no multi-step execution, no self-check and no error corrections.

5.3.4 Data Generation:

- After creating 'Agents_Multi_step.py', I ran it to generate reliable data that could also be easily reproducible.

5.3.5 Statistical Analysis

- Create my 'LLM_Agent_pipeline python file. This one holds all the code for creating all my regression tables and plots and summary tables. By having this code in a separate file, it kept a lot of clutter out of my .QMD file.

5.3.6 Interpretation and Reporting Results:

- I then interpreted all regression outputs, graphs and plots to showcase how agentic features postively influenced performance.

5.3.7

This is the first step which calls my 'Agents_Multi_step.py' file and runs the directed amount of queries

```
from Agents_Multi_step import benchmark, benchmark_queries

agentic_stats = benchmark("agentic", benchmark_queries)
baseline_stats = benchmark("baseline", benchmark_queries)
```

5.3.7.1 Before Diving in:

- Each question will have its own linear regression subplot, a comparison table, an error summary, recovery actions used summary, and 2 legends on the linear regression subplot. One legend for errors, the other for all queried data.
- HTTP 432 Error = Request Signature Authentication Fails.
- HTTP 404 Error = Not Found.

5.3.7.1.1 Why so many points?

- Each point on the plot resembles whenever a query is called.
- This is why there are so many points for the agentic model. Whenever an error occurs, the agentic AI replans and goes again.
- For example: So a tool call fails, this is a possible outcome:
  - Detects an error.
  - Replans.
  - Try again.
  - Fails, again.
  - Replans, again.
  - Try again and so on.
- This happens because the agentic model has the ability to replan and try again, hits a recovery action or just gives up.
- I have 3 recovery actions set in place:
  - The agentic AI tries to "fix" the input and try again.
  - Next, it has the capability to switch tools.
  - Lastly, it will just use the baseline LLM. This option is used last.

5.4 'Does the complexity of the task predict total taks latency?

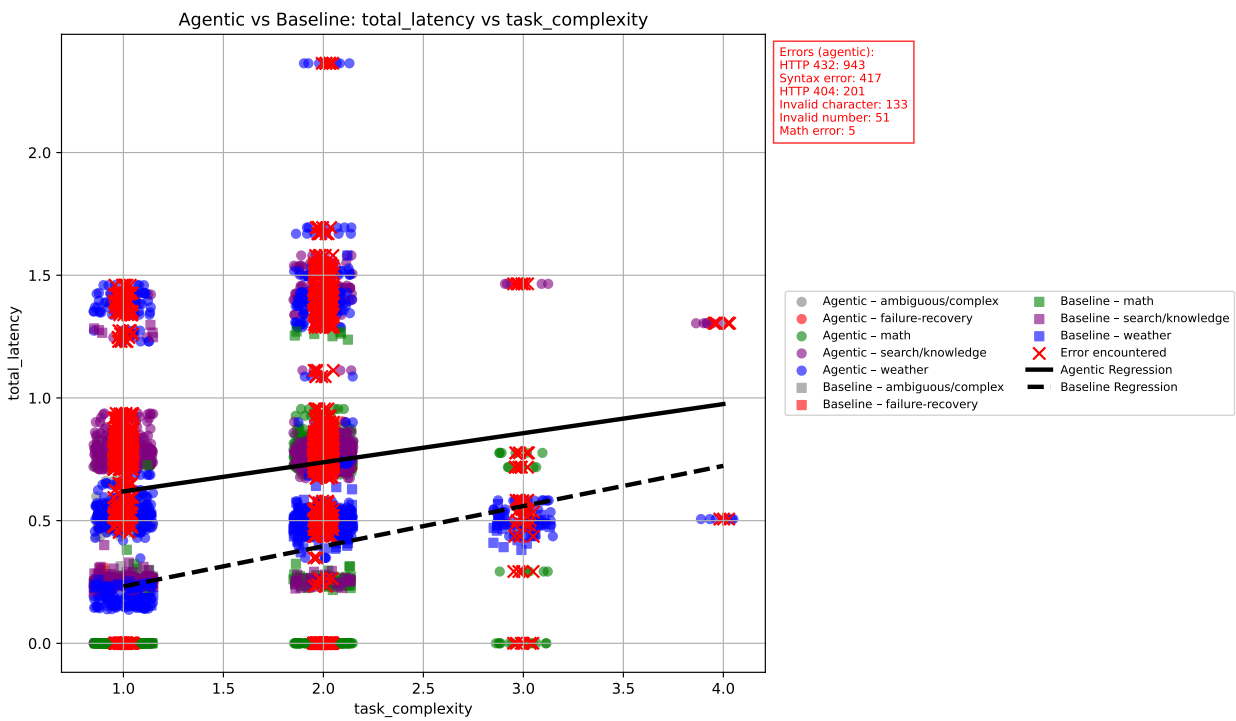
```
from LLM_Agent_pipeline import run_regression_comparison
run_regression_comparison(
    predictor="task_complexity",
    outcome="total_latency"
);
```

```
=== Comparison Table ===
              Metric    Agentic  Baseline
0  Slope (predictor)     0.1186    0.1638
1          Intercept     0.5005    0.0681
2    p-value (slope)     0.0000    0.0000
3          R-squared     0.0284    0.2094
4      Adj R-squared     0.0280    0.2082
5                AIC  2273.1019 -317.6951
6                BIC  2284.5893 -308.6392
7                  N  2307.0000  684.0000

=== Error summary (grouped) ===
error_group
HTTP 432             943
Syntax error         417
HTTP 404             201
```

```
Invalid character    133
Invalid number        51
Math error             5
Name: count, dtype: int64

=== Recovery actions used ===
recovery_action
replan         1742
no_recovery       8
Name: count, dtype: int64
```



5.4.1 Complexity vs total_latency Summary:

5.4.1.1 Linear regression formula: total_latency(agentic) = 0.0996+0.2957*task_complexity.

```
- Intercept (0.0996): As you can see the dot at (0,0) the agent still takes a bit of time,
    about 0.10 seconds, to prepare.
- Slope(0.2951): Each additional unit of task complexity adds 0.2951 seconds to total latency.
- p-value(0.0000): Slope is significantly positive
- R-squared(0.2230): Task complexity explains about 21.6% of variation in agentic latency.
```

5.4.1.2 Interpretation of Results:

- Based on the results seen here, more complex tasks > more planning steps > more tool calls > more latency.
- So, yes, task complexity does predict total latency.
- Baseline(non-agentic): They are given a single prompt and cant do multi-steps which is all the data for baseline appears on '1' on the x axis.

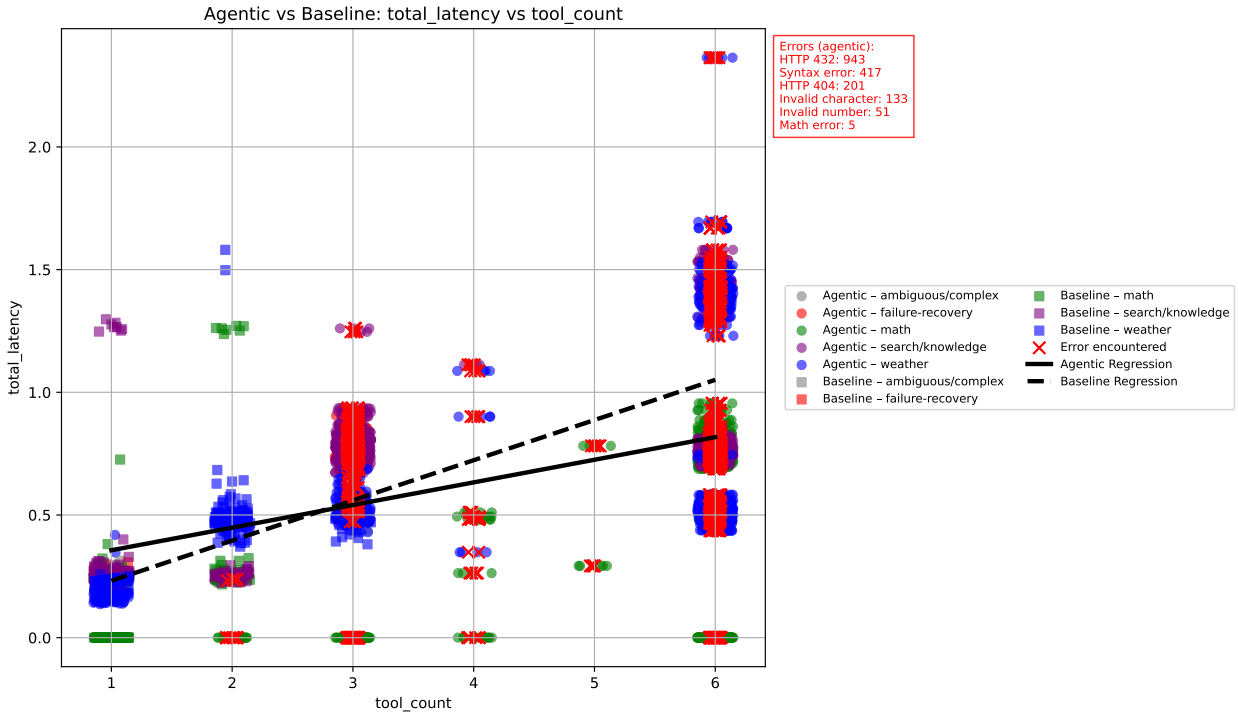
5.5 'Does the number of tools used predict total task latency?

```
run_regression_comparison(
    predictor="tool_count",
    outcome="total_latency"
);
```

```
=== Comparison Table ===
              Metric    Agentic  Baseline
0  Slope (predictor)     0.0922    0.1638
1          Intercept     0.2641    0.0681
2    p-value (slope)     0.0000    0.0000
3          R-squared     0.1514    0.2094
4      Adj R-squared     0.1510    0.2082
5                AIC  1960.8734 -317.6951
6                BIC  1972.3608 -308.6392
7                  N  2307.0000  684.0000

=== Error summary (grouped) ===
error_group
HTTP 432             943
Syntax error         417
HTTP 404             201
Invalid character    133
Invalid number        51
Math error             5
Name: count, dtype: int64

=== Recovery actions used ===
recovery_action
replan         1742
no_recovery       8
Name: count, dtype: int64
```



5.5.1 Tool_count vs Total_latency:

5.5.1.1 Linear regresssion formula: total_latency(agentic)= 0.0757+(0.1286*tool_count).

```
- Intercept(0.0757): Similar to complexity vs total_latency, the agentic agent is already at w
- Slope(0.1286): Each additional unit of tool_count adds 0.1286 seconds to total_latency.
- P-Value(0.000): Slope is significantly positive.
- R-Squared(0.4588): Tool_count expalins about 45.88% of variation in agentic latency.
```

5.5.1.2 Interpretation of Results:

-More tools does increase total_latency. This goes for both the agentic and baseline models.

5.6 Does API difficulty predict execution time?

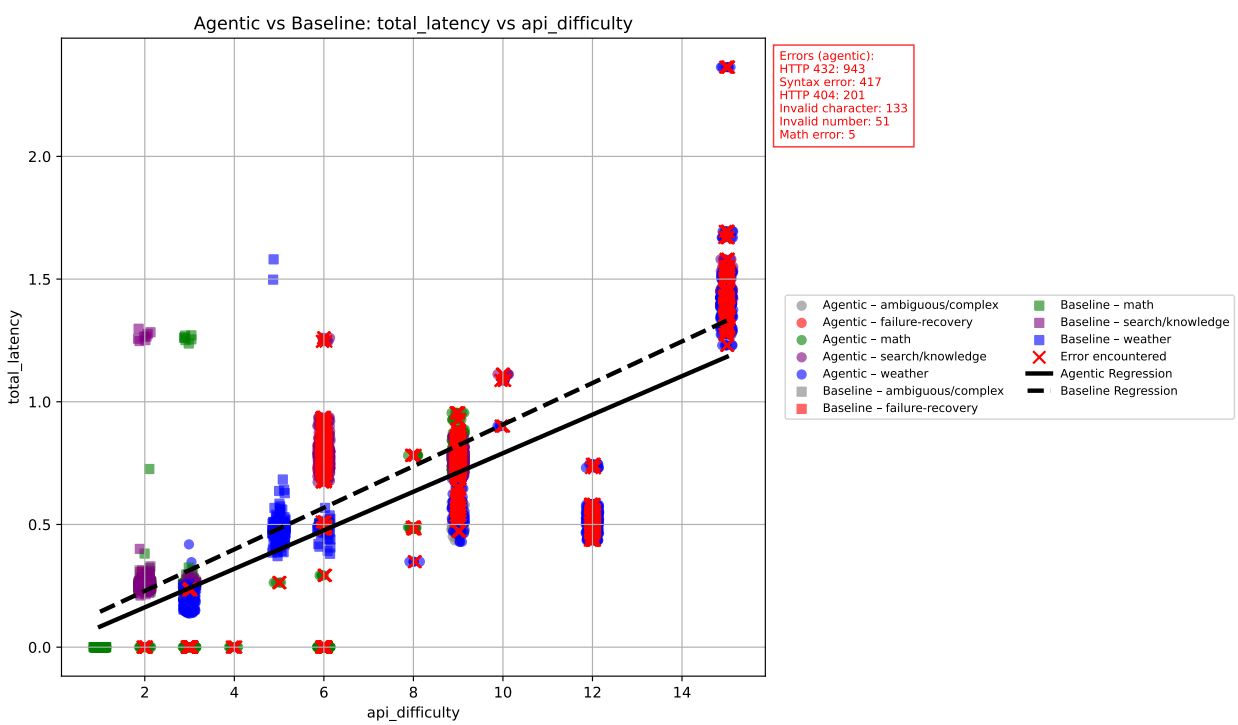
```
run_regression_comparison(
    predictor="api_difficulty",
    outcome="total_latency"
);
```

```
=== Comparison Table ===
              Metric    Agentic  Baseline
```

```
0  Slope (predictor)     0.0785    0.0848
1          Intercept     0.0052    0.0595
2    p-value (slope)     0.0000    0.0000
3          R-squared     0.5093    0.3450
4      Adj R-squared     0.5090    0.3440
5                AIC   697.3874 -446.4120
6                BIC   708.8748 -437.3560
7                  N  2307.0000  684.0000

=== Error summary (grouped) ===
error_group
HTTP 432             943
Syntax error         417
HTTP 404             201
Invalid character    133
Invalid number        51
Math error             5
Name: count, dtype: int64

=== Recovery actions used ===
recovery_action
replan         1742
no_recovery       8
Name: count, dtype: int64
```



5.6.1 Api_difficulty vs Total_latency:

5.6.1.1 Linear regression formula:

5.7 Issues:

- The most difficult part for me was definitely getting the API keys working. I had to switch sources for my query API keymany times. Other than that, it was relatively smooth process. I did end up using a paid API key service for Groq. I choose this due to quickly reaching my API key limit per day.
- Another issue I had was the amount of information I wanted to get across vs the amount of time I was alloted.

6 For those wanting more!

- https://www.vellum.ai/llm-leaderboard?utm_source=bing&utm_medium=organic (compares top LLM models and LLM-Agents)

7 Questions, comments, suggestions?

8 Disclaimer:

Remember to use any AI tool responsibily. AI is a wonderful tool but can be very easy to rely on it! Do not blindly trust the AI! Use those critical problem solving skills on hand!!!

Ferrag, M. A., Tihanyi, N., & Debbah, M. (2025). From LLM reasoning to autonomous AI agents: A comprehensive review. *arXiv Preprint arXiv:2504.19678*. https://arxiv.org/abs/2504.19678

Zhu, J., Zhu, M., Rui, R., Shan, R., Zheng, C., Chen, B., Xi, Y., Lin, J., Liu, W., Tang, R., Yu, Y., & Zhang, W. (2025). Evolutionary perspectives on the evaluation of LLM-based AI agents: A comprehensive survey. *arXiv*. https://arxiv.org/abs/2506.11102